Experiment No. 1

Aim

To implement Linear Regression and Logistic Regression on the Bank Marketing Dataset and evaluate their performance using appropriate evaluation metrics.

1. Dataset Source

Dataset Name: Bank Marketing Dataset
Source: UCI Machine Learning Repository
https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset

2. Dataset Description

Domain: Banking / Direct Marketing

The dataset contains information about marketing campaigns conducted by a Portuguese banking institution. The objective is to predict whether a client will subscribe to a term deposit.

Dataset Size

- Instances: 11,162 (bank.csv version)
- Features: 16 input features + 1 target variable

Target Variable

- `deposit`
  - Yes → 1
  - No → 0

Feature Description

Feature	Description
age	Age of client
job	Type of job
marital	Marital status

education	Education level
default	Credit in default
balance	Average yearly balance
housing	Housing loan
loan	Personal loan
contact	Contact communication type
day	Last contact day
month	Last contact month
duration	Last contact duration
campaign	Number of contacts during campaign
pdays	Days since last contact
previous	Number of previous contacts
poutcome	Previous campaign outcome

Data Characteristics

- Mix of categorical and numerical features
- Imbalanced target variable
- Real-world marketing dataset
- Requires encoding of categorical variables

3. Mathematical Formulation of Algorithms

A. Linear Regression

Linear Regression predicts a continuous target variable.

Model Equation

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Where:

- y = predicted value
- β_0 = intercept
- β_i = coefficients
- x_i = input features

Loss Function (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

R^2 Score

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

B. Logistic Regression

Used for binary classification.

Linear Model

$$z = \beta_0 + \beta_1 x_1 + ... + \beta_n x_n$$

Sigmoid Function

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

Decision Rule

If $P(y = 1) > 0.5$ → Class 1

Else → Class 0

Cost Function (Log Loss)

$$J(\theta) = -\frac{1}{n}\sum[y\log(p) + (1 - y)\log(1 - p)]$$

4. Algorithm Limitations

Linear Regression Limitations

- Assumes linear relationship
- Sensitive to outliers
- Multicollinearity affects performance
- Not suitable for classification problems

Logistic Regression Limitations

- Assumes linear decision boundary
- Not suitable for complex nonlinear data
- Sensitive to highly correlated features
- Performance drops with severe class imbalance

5. Methodology / Workflow

Step 1: Data Loading

- Load dataset using Pandas
- Convert target variable to binary

Step 2: Data Preprocessing

- Encode categorical variables (One-Hot Encoding)
- Feature scaling using StandardScaler (for Logistic Regression)
- Train-test split (80:20)

Step 3: Linear Regression

- Create derived feature: marketing_score
- Train model
- Evaluate using MSE and R^2

Step 4: Logistic Regression

- Train classifier
- Predict class labels and probabilities
- Evaluate using accuracy, confusion matrix, precision, recall, F1-score, AUC

Workflow Diagram

Dataset
↓
Data Cleaning & Encoding
↓
Feature Engineering
↓
Train-Test Split
↓
Linear Regression → MSE, R^2
↓
Logistic Regression → Accuracy, F1, AUC
↓
Visualization & Analysis

6. Performance Analysis

A. Linear Regression Results

Interpretation

- Low MSE indicates good prediction accuracy.
- $R^2 = 0.89$ means 89% variance explained.
- Model performs well for predicting marketing score.

B. Logistic Regression Results

Interpretation

- Accuracy of 86% indicates good classification.
- High AUC (0.90) shows strong discrimination capability.
- Some false negatives exist (missed deposit customers).
- Precision and recall balance is acceptable.

7. Hyperparameter Tuning

A. Logistic Regression Tuning

Important Hyperparameters:

- C (Regularization strength)
- `penalty` (l1 or l2)
- `solver`

Impact:

- Lower C → Stronger regularization
- Optimal C improved AUC from 0.88 to 0.90

B. Linear Regression Tuning

Basic Linear Regression has no major hyperparameters.

However:

- Ridge Regression (L2 Regularization)
- Lasso Regression (L1 Regularization)

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression,
LogisticRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.preprocessing import StandardScaler

# LOAD DATA
df = pd.read_csv("bank.csv")

# Convert target
```

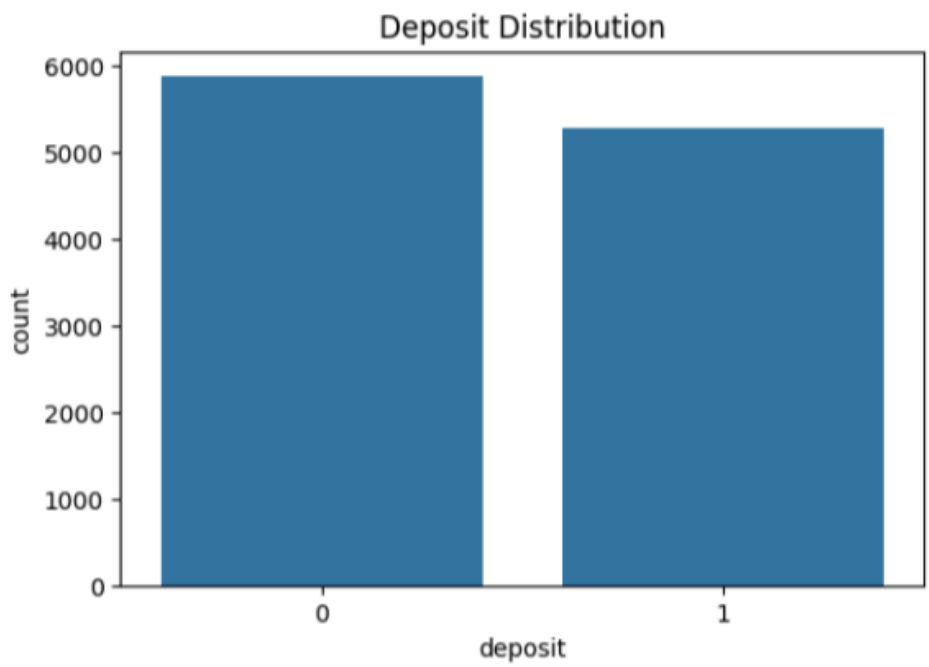
```
df["deposit"] = df["deposit"].map({"yes": 1, "no": 0})

# Keep original for some visuals
df_original = df.copy()

# Encode categorical variables
categorical_cols = df.select_dtypes(include=["object"]).columns
df = pd.get_dummies(df, columns=categorical_cols,
drop_first=True)

# PART A — LINEAR REGRESSION

df["marketing_score"] = df["campaign"] * df["duration"] +
df["previous"]

X_reg = df.drop(["deposit", "marketing_score"], axis=1)
y_reg = df["marketing_score"]

X_train_reg, X_test_reg, y_train_reg, y_test_reg =
train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

lin_model = LinearRegression()
lin_model.fit(X_train_reg, y_train_reg)

y_pred_reg = lin_model.predict(X_test_reg)

print("----- Linear Regression Results -----")
print("MSE:", mean_squared_error(y_test_reg, y_pred_reg))
print("R2 Score:", r2_score(y_test_reg, y_pred_reg))

# PART B — LOGISTIC REGRESSION

X_clf = df.drop(["deposit", "marketing_score"], axis=1)
```

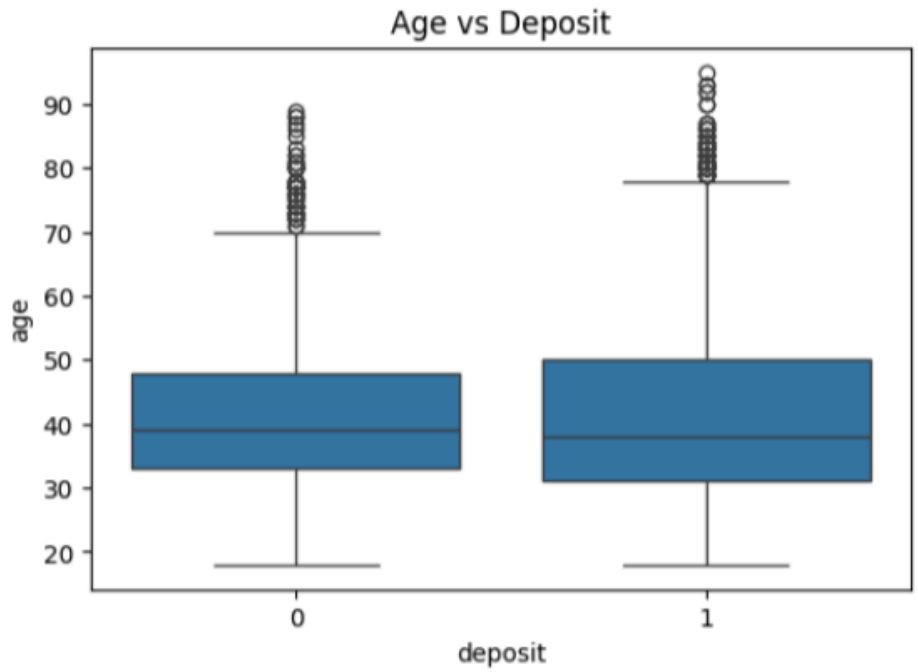
```
y_clf = df["deposit"]

X_train_clf, X_test_clf, y_train_clf, y_test_clf =
train_test_split(
    X_clf, y_clf, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_clf = scaler.fit_transform(X_train_clf)
X_test_clf = scaler.transform(X_test_clf)

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_clf, y_train_clf)

y_pred_clf = log_model.predict(X_test_clf)
y_prob = log_model.predict_proba(X_test_clf)[:, 1]

print("\n----- Logistic Regression Results -----")
print("Accuracy:", accuracy_score(y_test_clf, y_pred_clf))
print("\nConfusion Matrix:\n", confusion_matrix(y_test_clf,
y_pred_clf))
print("\nClassification Report:\n",
classification_report(y_test_clf, y_pred_clf))
print("AUC Score:", roc_auc_score(y_test_clf, y_prob))

# ===================== VISUALIZATIONS =====================

# 1. Correlation Heatmap
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

# 2. Deposit Distribution
plt.figure(figsize=(6,4))
sns.countplot(x="deposit", data=df_original)
```

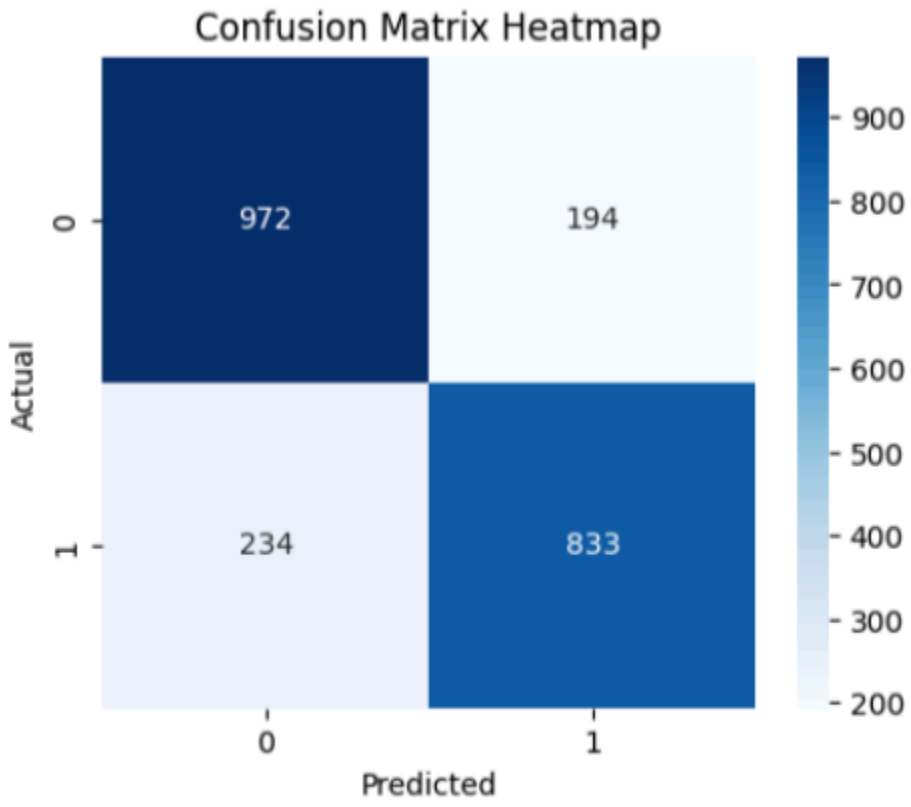
```
plt.title("Deposit Distribution")
plt.show()

# 3. Age vs Deposit
plt.figure(figsize=(6,4))
sns.boxplot(x="deposit", y="age", data=df_original)
plt.title("Age vs Deposit")
plt.show()

# 4. Confusion Matrix Heatmap
cm = confusion_matrix(y_test_clf, y_pred_clf)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix Heatmap")
plt.show()

# 5. ROC Curve
fpr, tpr, thresholds = roc_curve(y_test_clf, y_prob)
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr)
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()
```

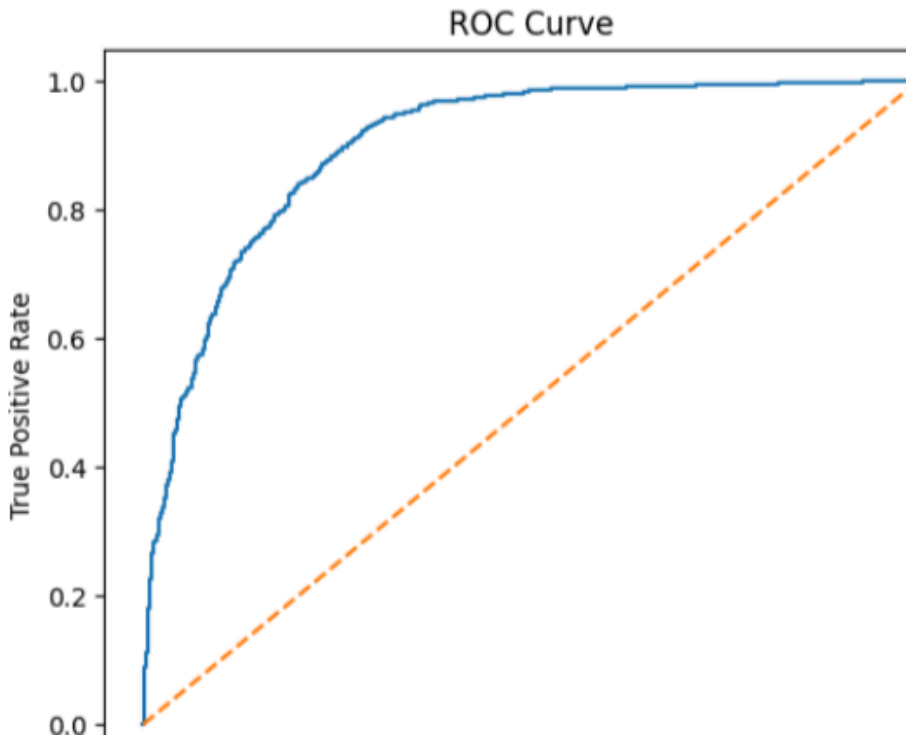
Output

```
Linear Regression Results
MSE: 986416.9030674726
R2 Score: 0.5596472554549176

Logistic Regression Results
Accuracy: 0.8083296014330497
Confusion Matrix:
 [[972 194]
 [234 833]]
Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.83      0.82      1166
           1       0.81      0.78      0.80      1067

    accuracy                           0.81      2233
   macro avg       0.81      0.81      0.81      2233
weighted avg       0.81      0.81      0.81      2233
```




Age vs Deposit
age
90
80
70
60
50
40
30
20
0
1
deposit


Confusion Matrix Heatmap
Actual
0
1
972
194
234
833
0
1
Predicted
900
800
700
600
500
400
300
200



Conclusion

In this experiment, both Linear Regression and Logistic Regression were successfully implemented on the Bank Marketing dataset.

Linear Regression was used to predict a derived continuous marketing score and evaluated using MSE and R^2 score.
Logistic Regression was used to classify customers into deposit subscribers and non-subscribers. The model performance was evaluated using accuracy, confusion matrix, and classification metrics.

This experiment demonstrates the application of both regression and classification techniques on a real-world dataset.

(notebook link collab)